GPU PROGRAMMING

CUDA:

Architecture, Memory & Kernel Optimization

A deep dive into how GPUs actually work — and how to write fast code for them

Alessandro Bartolocci · Sapienza University of Rome · 2026

ABOUT ME

Alessandro Bartolocci

BSc — Mathematical Sciences for Artificial Intelligence

Sapienza University of Rome

Strong interest in mathematics, programming, AI and quantitative finance. Curious about physics and other sciences. Currently focused on:

- Parallel programming
- Kernel optimization
- GPU architecture

linkedin.com/in/alessandro-bartolocci-1a12b9359

WHY THIS TALK

Most people use GPUs. Almost nobody understands them.

You've trained models on GPU

— but do you know why torch.cuda() makes things 10× faster?

You've called .to('cuda')

— but do you know what actually happens to your tensors in memory?

Your CUDA code is slow / you don’t know CUDA at all

This talk fills that gap

— hardware model → memory → five optimization patterns you can apply immediately.

ROADMAP

What we'll cover today

01 CPU vs GPU

two architectures, two philosophies

02 GPU Hardware

SMs, warps, tensor cores — what's on chip

03 CUDA Model

threads, blocks, grids — the execution model

04 Memory Hierarchy

registers → shared → L2 → global

05 5 Optimizations

coalescing · divergence · latency · occupancy · bank conflicts

06 Putting it together

CPU vs GPU

01

Two different philosophies

Latency-optimised vs. throughput-optimised — understanding this distinction changes how you write every single CUDA kernel.

Spoiler: neither is better. They're designed for completely different problems.

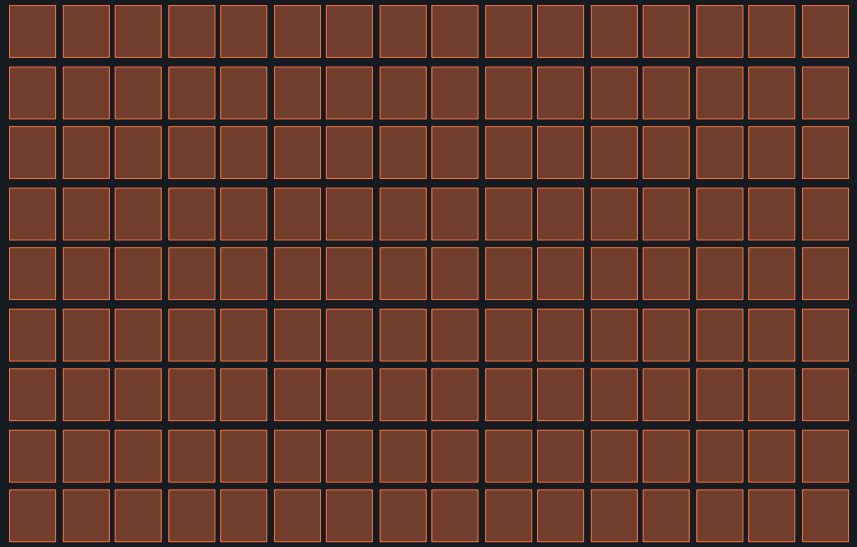
THE ARCHITECTURAL SPLIT

Few powerful cores vs. thousands of simple ones

CPU — Intel Core i9

8 complex cores · big cache · branch predictor · out-of-order exec

GPU — NVIDIA H100



16,896 CUDA cores · simple ALUs · optimised for throughput

THE CORE TRADEOFF

Latency vs. Throughput

CPU — Latency optimised

Goal:	finish ONE task as fast as possible
Cores:	8–24 powerful, out-of-order cores
Cache:	huge L3 (up to 96MB) to minimise stalls
Branch predictor:	aggressively speculates to avoid waiting
Great for:	OS scheduling, databases, web servers, game engines

"How fast can I finish this task?"

GPU — Throughput optimised

Goal:	finish MANY tasks at the same time
Cores:	thousands of small, in-order cores
Cache:	small per-core; hide latency with more threads
No branch predictor:	divergence is expensive — avoid branches
Great for:	matrix ops, image processing, simulations, ML

"How many tasks can I finish per second?"

THE KEY INSIGHT

A GPU hides latency through parallelism.

When one warp stalls waiting for memory,
the GPU instantly switches to another warp.

The CPU fights latency with caches and prediction.
The GPU drowns latency in work.

This single idea explains every optimization we'll cover today.

HISTORICAL BACKGROUND

From Doom to the Transformer

How the GPU went from pixels to neurons

— 1993 → 2024

02

TIMELINE

30 years that changed everything

1993
Doom releases on PC

1999
NVIDIA releases GeForce 256

2006
CUDA is released: GPU is now programmable

2012
AlexNet wins against ImageNet

2017
Transformer & Attention

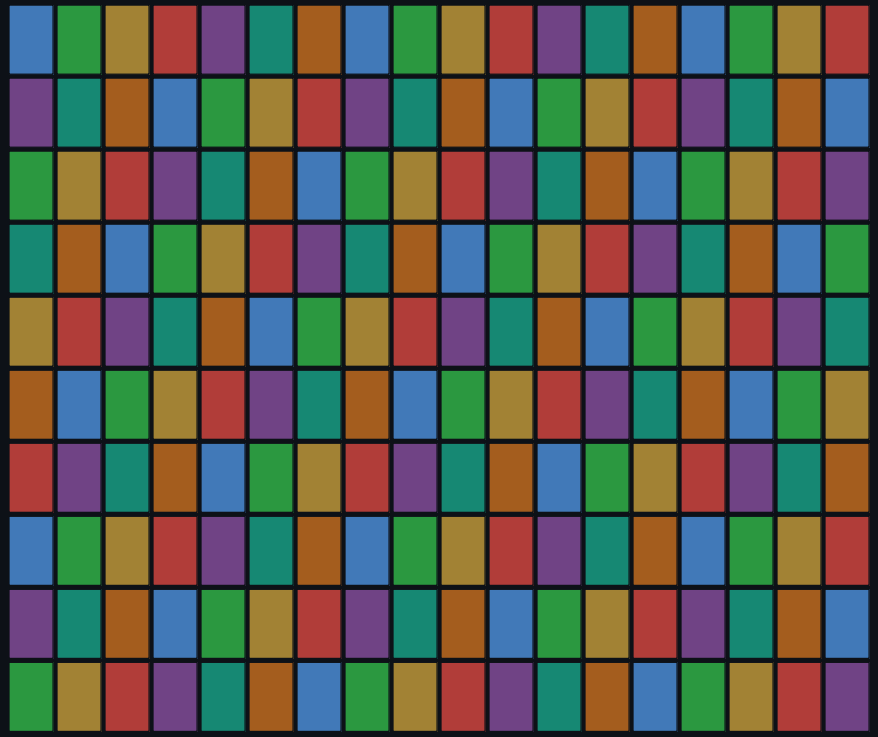
1993 – 1999

It all starts with videogames

The original problem:

Rendering a 3D frame means calculating the color of hundreds of thousands of pixel... All together.

Each pixel is independent → natural parallelism



Each pixel = an independent computation

2006 — AHA MOMENT

NVIDIA releases CUDA

> *"What if I could use GPU not only for gaming,*
> *But also for every parallel computation?*

programmable GPU

Not only pixel — every type of data

New programming language

C-like, developers are used to that syntax

open ecosystem

Whoever can use GPU power

2012 — THE TURINIG POINT

AlexNet wins against ImageNet Everything changes

What happened

- AlexNet: neural network trained on GPU
- Classification Error: 15.3%
- Second candidate: 26.2%
- Win margin: -11%
- ML community doesn’t sleep for weeks

The real insight

Researchers understood that artificial neurons, like pixels, are highly parallelizable because of independency between them:

Matmul is highly parallelizable

The GPU was no longer just a tool for gaming — it was the engine of the future.

THE KEY CONNECTION

Pixel and neurons: both independent

VIDEOGIOCHI

Each pixel →
Calculate R,G,B
Apply texture
Apply light
= independent operation

1920×1080 = 2M pixel in parallel

=

DEEP LEARNING

Each neuron →
Get input
Multiply by weight
Apply function
= independent operation

GPT-3: 175B neurons in parallel

NVIDIA GPU HARDWARE

02

What's actually on the chip

GPC → TPC → SM — a hierarchy designed around data parallelism at every level.

GPC | TPC | SM | Warp | Tensor Core | Memory

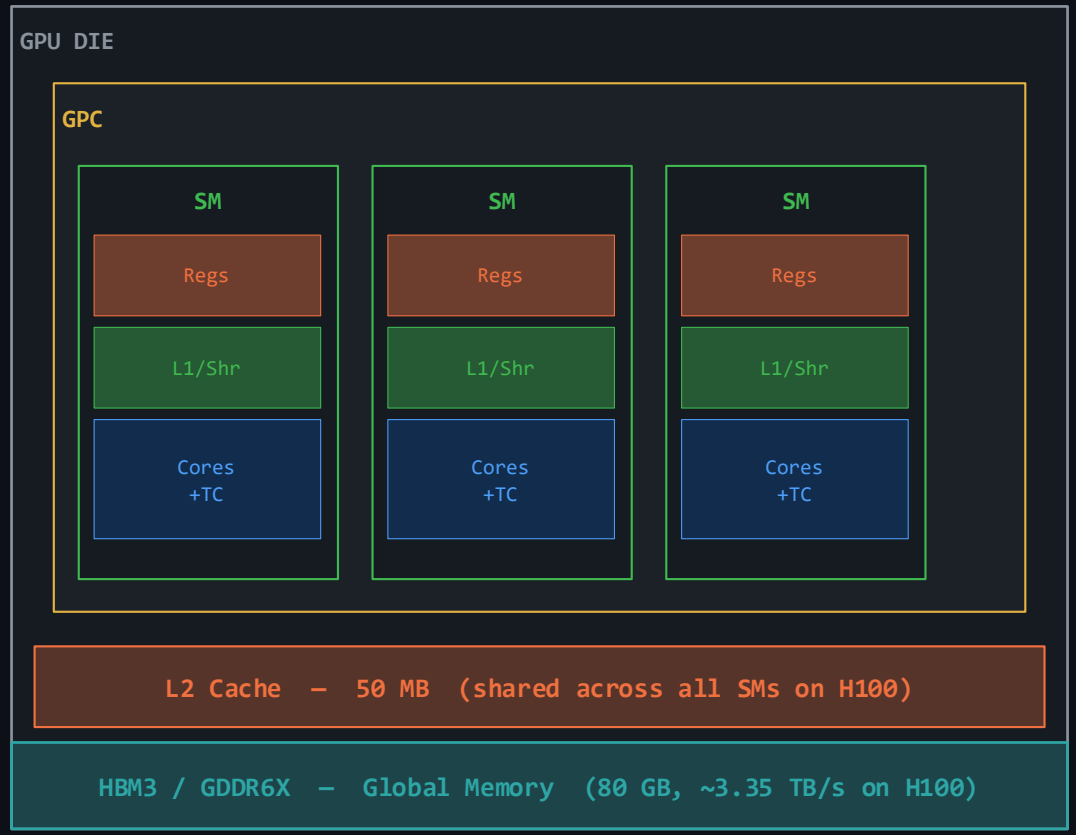
THE CHIP HIERARCHY

GPU → GPC → TPC → SM → Warp → Thread

GPU *the whole chip*	Contains all compute, memory controllers, NVLink ports, PCIe interface.
GPC *Graphics Processing Cluster*	Largest compute cluster. Has its own raster engine (graphics). Groups multiple TPCs.
TPC *Texture Processing Cluster*	Contains 2 SMs + texture units. Shared texture cache and PolyMorph engine. Mostly a grouping unit today.
SM *Streaming Multiprocessor*	The real compute unit. Has CUDA cores, Tensor Cores, shared memory, L1/tex cache, register file, warp scheduler. This is where your kernel runs.
Warp *32 threads, same instruction*	32 threads executing in lockstep (SIMT). The GPU schedules at warp granularity — never individual threads.
Thread *one lane of execution*	One lane of a warp. Has its own registers. Executes one instruction per clock cycle (when not stalled).

GPU MEMORY — WHERE THINGS LIVE

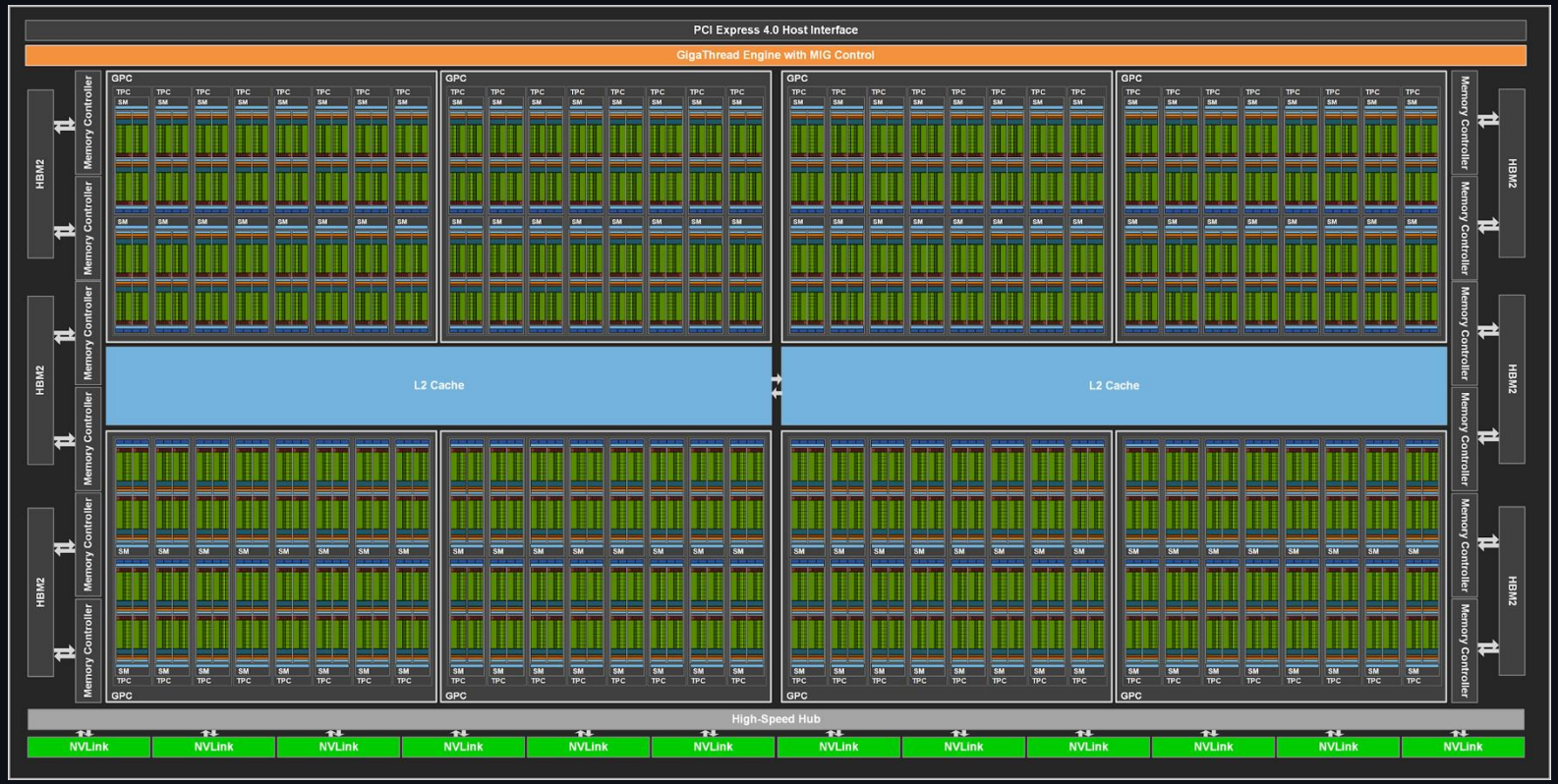
Physical location determines speed, scope and lifetime



Memory	Latency	BW
Registers *per thread*	~1 cy	~20 TB/s
Shared Mem *per block (SM)*	~20–30 cy	~12 TB/s
L1 Cache *per SM*	~20-30 cy	~12 TB/s
L2 Cache *whole chip*	~200-400 cy	~5 TB/s
Global (HBM) *whole chip*	~600-800 cy	3.35 TB/s
Constant Mem *all (RO)*	~20 cy (cached)	cached

GPU MEMORY — WHERE THINGS LIVE

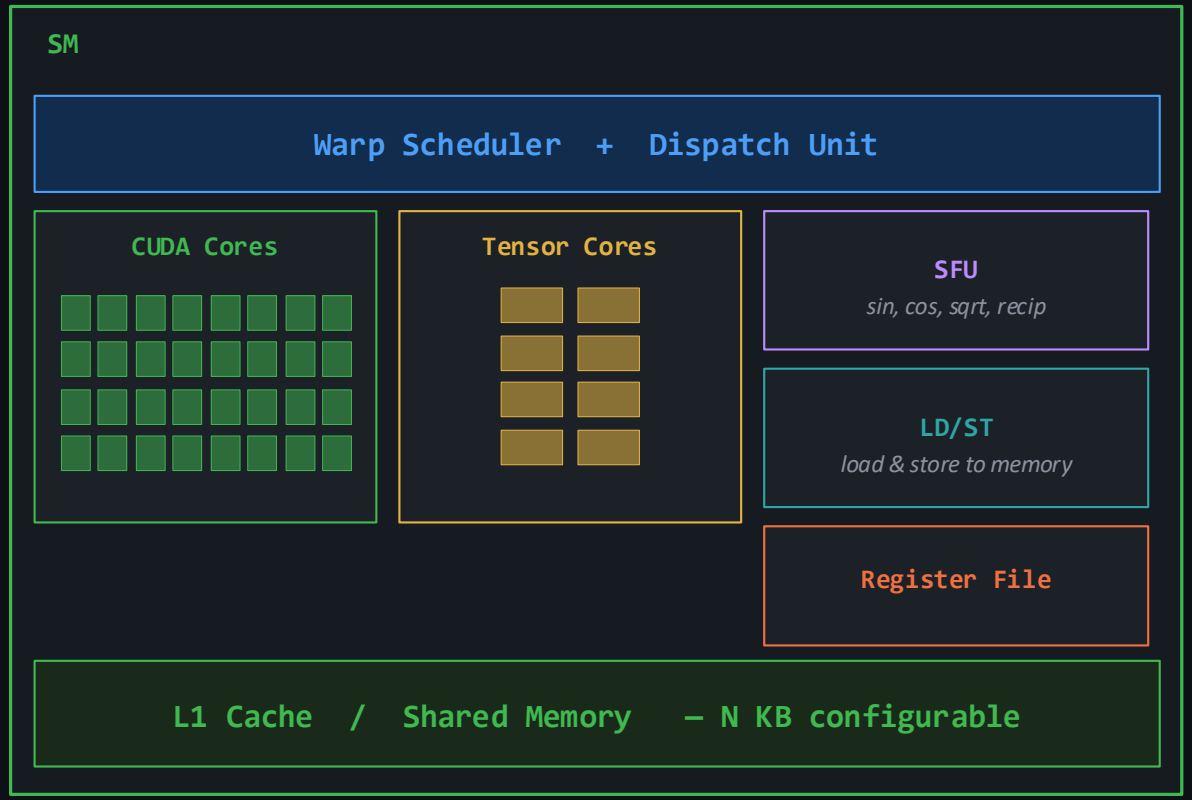
Physical location determines speed, scope and lifetime



Ampere architecture

INSIDE ONE SM (generic)

The Streaming Multiprocessor — anatomy



Warp Scheduler

Picks which warp runs each clock. Keeps the SM busy while others stall on memory.

CUDA Cores (FP32)

General-purpose floating point. Your add/mul/fma operations.

Tensor Cores

Tensor Cores speed up small-tile matrix multiply-accumulate operations in NVIDIA GPUs, greatly accelerating deep learning workloads.

SFU

Hardware transcendentals. Much faster than software, but limited units, so transcendental-heavy code can bottleneck here.

LD/ST + L1/Shared

Handle memory accesses inside an SM, issuing loads and stores for threads. L1 cache and shared memory are located within the SM and are tightly coupled to its execution pipelines

The Streaming Multiprocessor — anatomy

Ampere architecture SM

THE WARP — GPU'S BASIC UNIT OF SCHEDULING

32 threads. Same instruction. Every clock cycle.

T0	T1	T2	T3	T4	T5	T6	T7	T8	T9	T10	T11	T12	T13	T14	T15
T16	T17	T18	T19	T20	T21	T22	T23	T24	T25	T26	T27	T28	T29	T30	T31

→ All 32 threads execute: FMA R0, R1, R2 (same instruction, different data)

SIMT model

Single Instruction, Multiple Threads. All 32 lanes run in lockstep (though since Volta the model is more flexible). No per-thread instruction fetch.

Warp latency hiding

The SM keeps many warps ready, and when one stalls (e.g. memory load), another ready warp is scheduled.

Divergence cost

If threads in a warp take different if/else branches, the GPU serialises both paths. A good part of your throughput, gone.

Occupancy

More resident warps = better latency hiding. Register and shared memory usage per thread limit how many warps fit per SM.

MEMORY — WHAT EACH LEVEL IS FOR

Choosing the right memory is the first optimization

Registers

live variables for a thread

Too many regs per thread → fewer warps fit per SM → lower occupancy.

Shared Memory

fast scratchpad shared by the whole thread block

Misaligned access → bank conflicts → serialised reads. The #1 shared memory pitfall.

L1 Cache

automatic cache for global memory reads

Shares physical SRAM with shared memory. More shared mem = less L1.

L2 Cache

chip-wide cache for global memory

50 MB on H100. Locality across blocks? L2 will catch it.

Global Memory (HBM)

main GPU DRAM — large but slow

Every uncoalesced access wastes bandwidth. Rule: threads in a warp must access consecutive addresses.

Constant Memory

read-only, broadcast-cached data

If all 32 threads in a warp read the SAME address → single broadcast. Otherwise serialised.

THE CUDA PROGRAMMING MODEL

03

Thread. Block. Grid. How you talk to the GPU.

The programming model is the bridge between your code and the hardware.

`Host (CPU) → launches kernel → Device (GPU) → executes threads`

HOST vs DEVICE

Two separate processors. Two separate memory spaces.

HOST (CPU)

Runs serial code
main(), data loading, logic

Allocates GPU memory
cudaMalloc(&d_ptr, size)

Copies data to GPU
cudaMemcpy(..., H2D)

Launches kernels
kernel<<<grid, block>>>(...)

Copies results back
cudaMemcpy(..., D2H)

Frees GPU memory
cudaFree(d_ptr)

PCIe / NVLink

DEVICE (GPU)

Runs kernel code
__global__ void myKernel(...)

Each thread has an ID
threadIdx, blockIdx, blockDim, gridDim

Threads grouped in warps
32 threads, SIMT execution

Warps grouped in blocks
up to 1024 threads per block

Blocks grouped in grid
up to 2^31-1 blocks per dim

No shared state between blocks
blocks can run in any order, on any SM

THREAD

The single worker

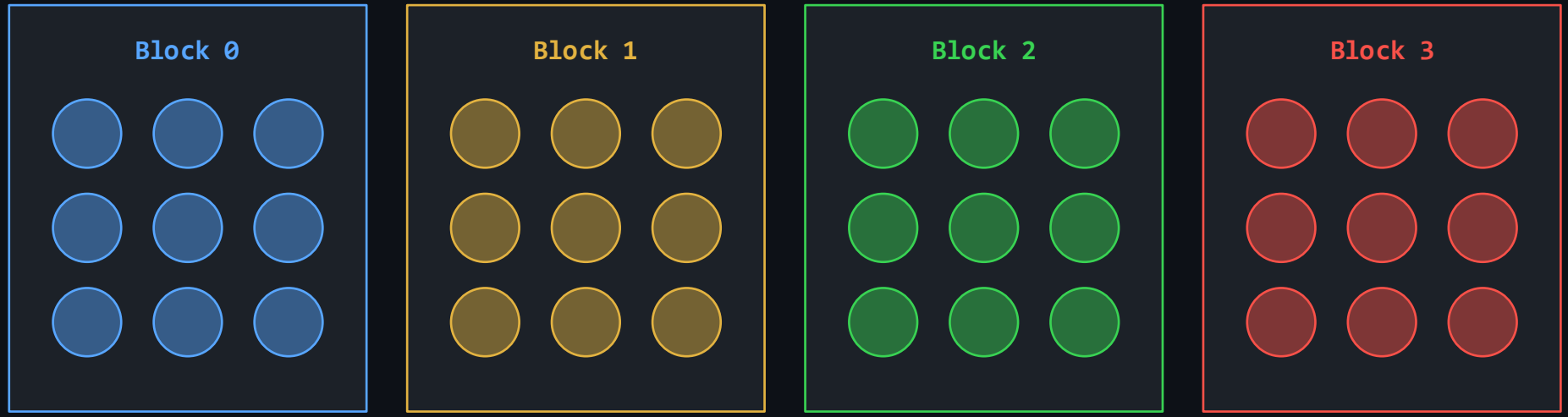
The thread is the smallest work unit
In the CUDA model.

- ⚡ Executes a single instance of the kernel,
- 🔢 Has his own ID — knows “where" he is
- 🔄 Thousands run in parallel

BLOCK

A group of workers

A block is a group of threads that works together and can communicate.



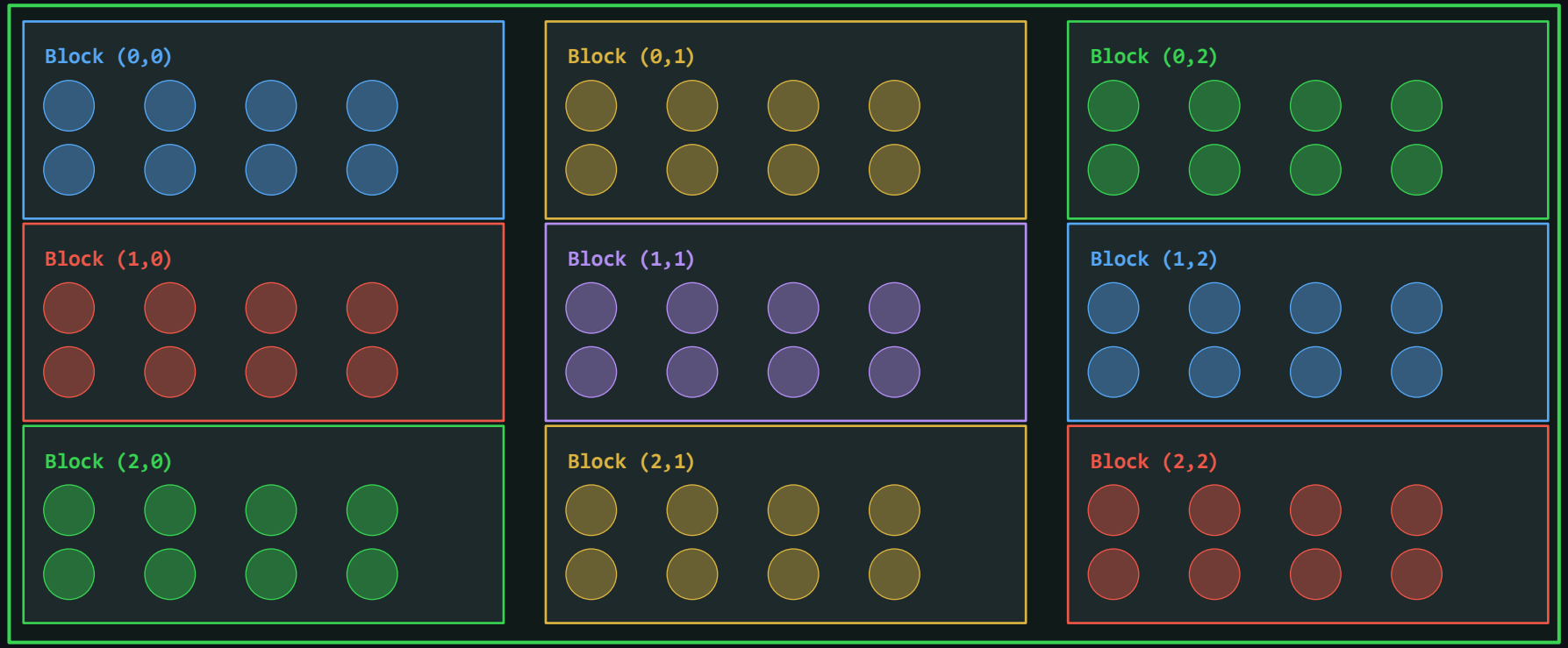
Thread in the same block can see a common fast memory (shared memory)

→ *This is one of the key optimizations*

GRID

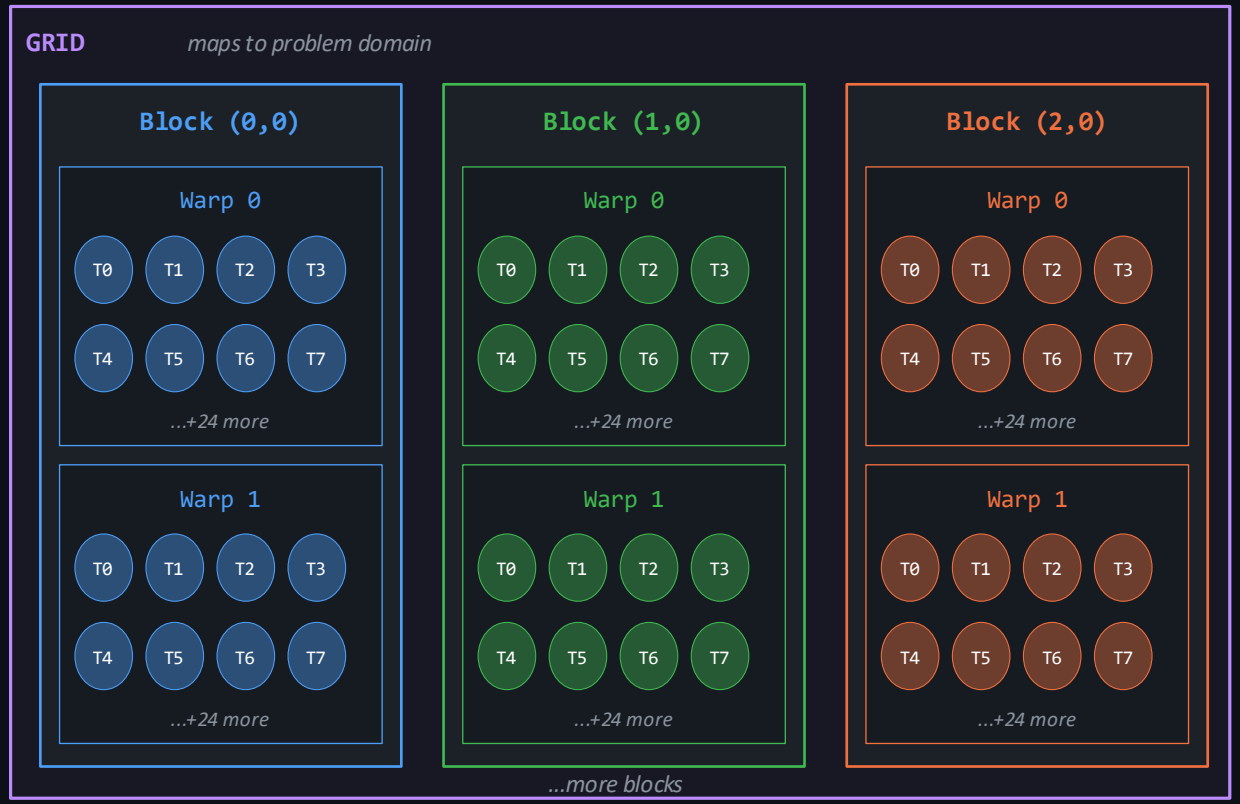
All the groups together

The Grid is the set of all blocks launched to solve the problem.



THE WHOLE THREAD HIERARCHY

Grid → Block → Warp → Thread



Thread

Has his ID. Owns its registers. The smallest unit of work.

Block

1–1024 threads. Runs on ONE SM. Threads can sync (__syncthreads()) and share memory.

Warp

32 consecutive threads from a block. Scheduled together. SIMT execution.

Grid

All blocks for one kernel launch. Blocks are independent — can run in any order on any SM.

How a thread knows what data it owns

Kernel launch syntax

```
kernel <<< gridDim, blockDim >>> ( args... )
```

blocks in grid
e.g. dim3(256,1,1)

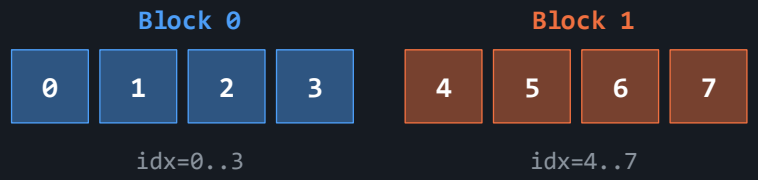
threads per block
e.g. dim3(4,1,1)

Global thread index (1D)

```
int idx = blockIdx.x * blockDim.x
          + threadIdx.x;
```

→ *maps thread to one element of your array*

Visualised (4 threads/block)



Always check bounds: if (idx < N) — grid size is often rounded up to a multiple of blockDim.

KERNEL EXECUTION LIFECYCLE

What actually happens when you launch a kernel

1

CPU side

Kernel launch

CPU calls kernel<<<grid,block>>>(). This is asynchronous — CPU continues immediately. The launch puts work into the GPU command queue.

2

GPU scheduler

Block dispatch

GPU's GigaThread Engine assigns blocks to free SMs. A block stays on its SM until all threads finish. No preemption mid-block.

3

SM

Warp creation

SM splits the block into warps of 32. All warps are eligible to run. Warp scheduler picks one warp per clock to issue to the execution units.

4

Warp scheduler

Warp execution (SIMT)

Selected warp issues one instruction across all 32 lanes simultaneously. If a warp stalls (memory, sync), another ready warp takes its turn instantly.

5

SM + memory

Memory ops

LD/ST units fire off memory requests. Global memory takes ~600 cycles. Warp stalls; scheduler switches to a different warp. This is latency hiding.

6

CPU sync

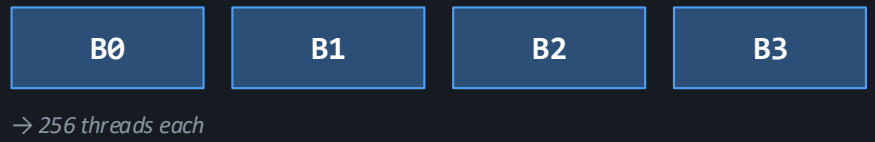
cudaDeviceSynchronize()

CPU blocks until all GPU work is done. Without this, reading results from CPU-side before the kernel finishes gives garbage data.

Grid design is the first CUDA decision you make

1D — vector of N=1024 elements, 256 threads/block

gridDim = ceil(1024/256) = 4 blocks



2D — matrix 1024×1024, block size 32×32 threads

gridDim = (32×32) blocks → 1,048,576 threads total

Rules: blockDim multiple of 32 · blockDim ≤ 1024 · prefer 128 or 256 threads/block as starting point

Controlling execution order

Synchronisation primitives

`__syncthreads()`

block-level
Barrier — all threads in a block must reach this point before any can proceed. Use after writing to shared memory.

`cudaDeviceSynchronize()`

host-level
CPU waits until ALL GPU work on ALL streams is done. Expensive — use only when you must read results on CPU.

`cudaStreamSynchronize(s)`

stream-level
CPU waits for all work in one stream. More fine-grained than device sync.

`cudaEventSynchronize(e)`

event-level
CPU waits until a specific event (GPU timestamp) has been recorded. Used for precise timing and inter-stream deps.

CUDA Streams

A stream is a sequence of GPU operations that execute in order.
Operations in different streams can overlap.

Stream 0 (default):

H2D copy	kernel A	D2H copy

Stream 1:

H2D copy	kernel B

→ Stream 1's H2D copy overlaps with Stream 0's kernel A

Use streams to hide PCIe transfer latency behind kernel execution.

Default stream 0 = serial. Custom streams = potential parallelism.

MODEL SUMMARY

What to keep in mind when writing a kernel

Each thread is one worker. Give it one element (or a small tile) to process.

Block size = multiple of 32. 128 or 256 is a safe default. Tune with the profiler.

Threads within a block can cooperate via __shared__ memory and __syncthreads().

Blocks cannot communicate with each other during a kernel. Use separate kernel launches.

Always compute: idx = blockIdx.x * blockDim.x + threadIdx.x. Always guard: if (idx < N).

The more warps you keep resident and busy, the more latency you hide. This is a key optimization.

KERNEL OPTIMIZATION

Five patterns. One mental model.

05

All five optimizations reduce to one thing: keep the hardware fed and moving.

- 01 Memory coalescing
- 02 Bank conflicts
- 03 Occupancy
- 04 Warp divergence
- 05 Vectorization

OPT 01

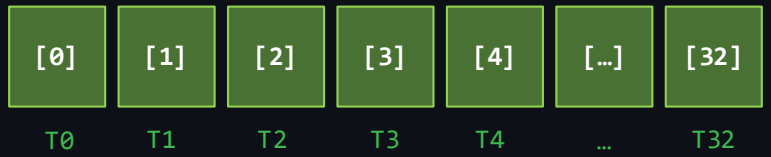
Memory Coalescing

The single most impactful optimization in GPU programming.

The rule:

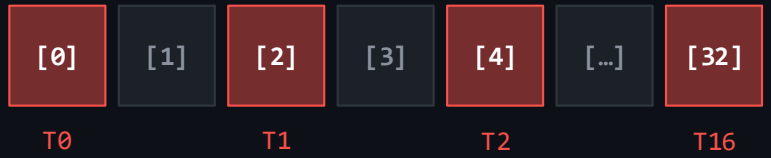
Threads in a warp must access consecutive memory addresses so the GPU can serve the entire warp in one transaction.

COALESCED ✓



→ 1 memory transaction for the whole warp

STRIDED (stride=2) ✗



→ 2 transactions, half bandwidth wasted

Worst case: random access → 32 separate 32-byte transactions per warp instead of 1 — 32× bandwidth overhead.

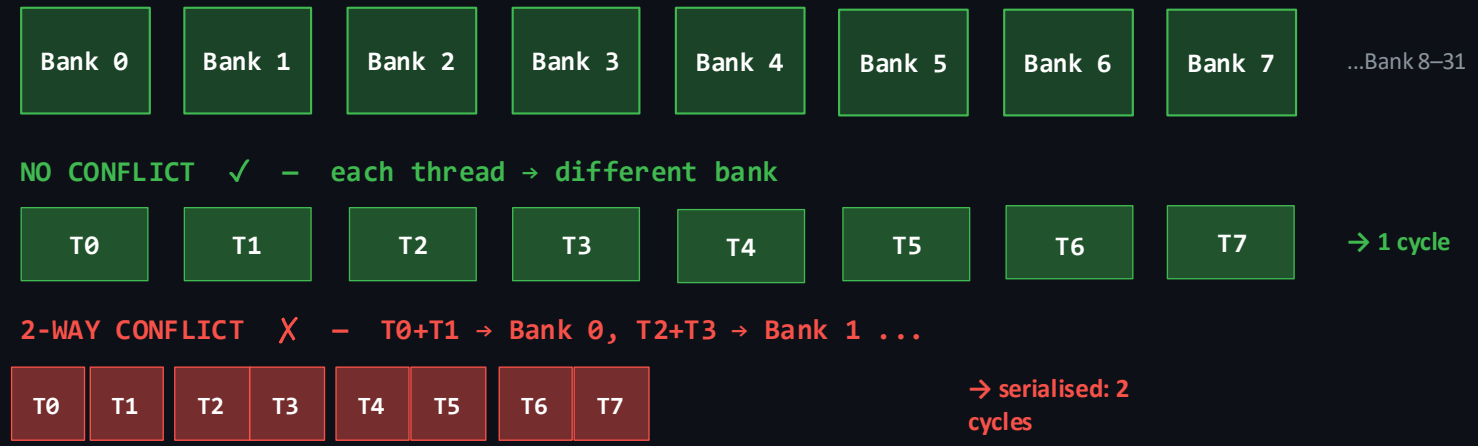
OPT 02

Shared Memory Bank Conflicts

Shared memory is fast — unless two threads in the same warp hit the same bank.

Shared memory is divided into 32 banks, each 4 bytes wide.

Address → Bank: `bank = (address / 4) % 32`



OPT 03

Occupancy

The fraction of maximum warps that are resident on an SM at any given time.

```
Occupancy = active warps per SM / maximum warps per SM
```

Why it matters:

When a warp stalls on memory (600 cycles), the SM switches to another ready warp.
High occupancy = more warps to switch to = latency hidden = ALUs never idle.

Occupancy limiters — the three resources that cap resident warps:

1. Registers per thread

H100 SM has 65,536 regs. Kernel using 64 regs/thread × 32 threads/warp = 2,048 regs/warp → max 32 warps. Use 128 regs/thread → max 16 warps.

2. Shared memory per block

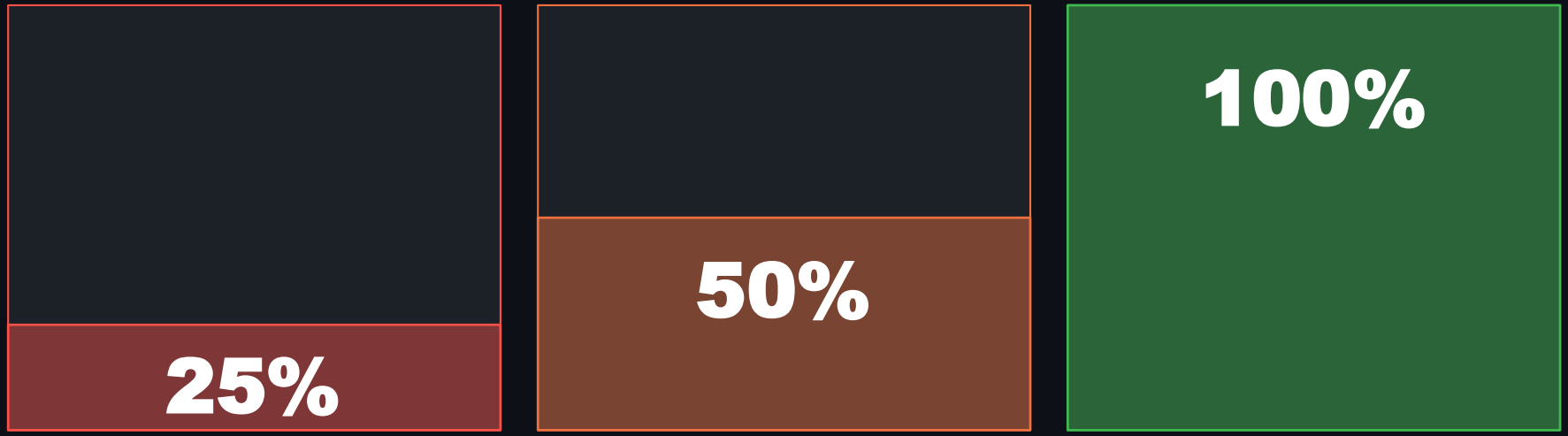
H100 SM has 228 KB shared. Kernel using 16 KB/block → max 14 blocks/SM → fewer resident warps.

3. Block size (threads/block)

Too small blocks → SM runs fewer total threads. Too large → resource pressure. Sweet spot: 128–256.

OPT 03 — OCCUPANCY IN PRACTICE

High occupancy ≠ fast. But low occupancy is almost always slow.



Low (25%)

128 regs/thread

48 KB/block

64 threads

Medium (50%)

64 regs/thread

24 KB/block

128 threads

High (100%)

32 regs/thread

8 KB/block

256 threads

Tool: Nsight Compute → SM Warp Occupancy chart shows exactly what's limiting you

OPT 04

Warp Divergence

When threads in the same warp take different paths, the GPU "serializes" them — throughput drops proportionally.

Recall: all 32 threads in a warp execute the SAME instruction each clock. If they diverge, the hardware masks off the inactive threads and runs each path separately. (From Volta onward, warps no longer follow strict lockstep execution. Thanks to independent thread scheduling, threads can diverge and make progress more independently, reducing the penalties of warp divergence)

Warp (T0–T31)

```
if (threadIdx.x < 16)
```

TRUE path (T0–T15)
path_A(); — 16 threads active, 16 masked

FALSE path (T16–T31)
path_B(); — 16 threads active, 16 masked

```
Reconverge at __syncwarp()
```

2 paths serialised → effective throughput 50% → avoid if/else that splits warps evenly

OPT 05

Vectorization

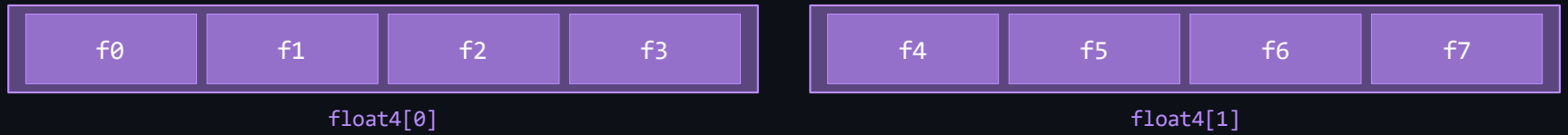
Load more data per instruction. Pack multiple values into one memory transaction.

GPU memory loads are coalesced in 32-byte or 128-byte transactions. Using float4 instead of float fills the transaction and halves the number of instructions per element.

SCALAR (float) — 1 value per load

8 loads × 4 bytes = 8 instructions, potentially 8 transactions

VECTORIZED (float4) — 4 values per load



2 loads × 16 bytes = 2 instructions — 4× fewer load instructions for same data

Vector types, alignment, and when to use them

CUDA vector types

int2	8 B	2× int32
int4	16 B	4× int32
float2	8 B	2× fp32
float4	16 B	4× fp32 ← most common
double2	16 B	2× fp64
half2	4 B	2× fp16, native Tensor Core input

Usage rules

Alignment

Pointer must be aligned to the vector size. float4* requires 16-byte alignment.

Access pattern

threadIdx.x indexes into float4 array. Thread 0 → float4[0], Thread 1 → float4[1]. Still coalesced — each float4 is 16 bytes, 32 threads → 512 bytes = 4 cache lines.

When to use

Bandwidth-bound kernels. If you're memory-bound (common!) and data is naturally multi-channel (RGB, xyz, NCHW), floatX reduces instruction count and often boosts throughput 1.5–2×.

When NOT to use

Compute-bound kernels — you're not waiting on memory

Benchmark: reinterpret_cast<float4*>(ptr) is the most common pattern — zero overhead, pure instruction count reduction.

The five patterns — when the profiler fires

01	**Memory coalescing** *Symptom: High global memory sector misses (Nsight)*	Ensure threads in warp access consecutive addresses. Prefer row access, check stride.
02	**Bank conflicts** *Symptom: High shared memory bank conflict counter*	Add +1 padding to shared arrays with power-of-2 stride. Check address pattern: (addr/4) % 32.
03	**Occupancy** *Symptom: Low SM warp occupancy, ALU underutilised*	Reduce registers per thread (__launch_bounds__), reduce shared memory per block, tune block size.
04	**Warp divergence** *Symptom: High warp execution efficiency loss*	Reorder data so warps process uniform types. Eliminate intra-warp branches or use predication.
05	**Vectorization** *Symptom: Bandwidth-bound, many small memory ops*	Use float4/int4. Ensure 16-byte alignment. Replace scalar loads with reinterpret_cast<float4*>.

CUDA AND DEEP LEARNING

05

Why neural networks love GPUs

From PyTorch to GPT-3: CUDA under the hood

— cuDNN · cuBLAS · PyTorch · TensorFlow

THE SECRET

A neural network is mostly made of matmuls!

Each computation is independent

→ perfect parallelism on GPU

COMPLETE STACK

When you write model.fit() what does really happen?

Your code	`model.fit(X, y)  # Python / PyTorch` *You write high level logic*
Framework	PyTorch / TensorFlow / JAX *Autograd and computational graphs*
CUDA Libraries	cuDNN · cuBLAS · cuFFT *Optimized kernels for conv, matmul, attention*
CUDA Runtime	Runs threads · allocates memory · sync *Handles the communication CPU ↔ GPU*

GPU HARDWARE — thousands of cores in

NVIDIA LIBRARIES

The secret engine under PyTorch and TensorFlow

cuBLAS

Basic Linear Algebra

Optimized matmul. Every fwd and bwd pass of a neural network does it thousands times.

Used by: PyTorch · TensorFlow · JAX

cuDNN

Deep Neural Network Library

Convolutions, attention, batch normalization — all the typical DL operations optimized.

Used by: Every existing DL framework

cuFFT

Fast Fourier Transform

Fourier Transformations on GPU. Used in audio AI, signal processing, and some diffusion models

Used by: Audio AI · Diffusion models

REAL NUMBERS

GPT-3: What's behind it

175B

Parameters

each parameter = mul on GPU

1.000+

GPU A100

Used for the training of GPT-3

34

Days of training

with 1.000 GPU in parallel

∞

Years on CPU

We won't have gpt3 in thousands of years from now

Without CUDA and GPUs: GPT-3 won't exist now.

PUTTING IT TOGETHER

06

How a fast CUDA kernel thinks

Every optimization we covered feeds into one workflow.
Here's what you check — in order — every time.

Profile first. Optimize second. Always.

THE OPTIMIZATION WORKFLOW

What you check — in this order — every time

Before writing code

1 Is this problem memory-bound or compute-bound?

Roofline model: peak FLOPS vs peak bandwidth. If arithmetic intensity < ridge point → you're memory-bound. Fix memory first.

Grid design

2 How do I map threads to data?

One thread per output element. Block size = 128 or 256 (multiple of 32). Grid = ceil(N / blockDim). Guard with if (idx < N).

Global memory

3 Are my global memory accesses coalesced?

threadIdx.x indexes fastest-moving dimension. Use SoA layout. Check Nsight: L1/TEX sector misses per warp > 1 is a red flag.

Shared memory

4 Am I reusing data within a block?

Load tile into shared memory once, reuse N times. Add +1 padding if stride is a power of 2. Sync with __syncthreads() after writes.

Warp efficiency

5 Are my warps diverging?

Check Nsight: Warp Execution Efficiency < 100% → divergence. Reorder data, remove intra-warp branches, use predication or branchless ops.

Occupancy

6 Am I keeping the SM busy?

Target 50–75% occupancy. Use __launch_bounds__ to cap registers. Reduce shared memory per block. Profile with Nsight SM Warp Occupancy.

Instruction throughput

7 Am I wasting instructions on loads?

Vectorize with float4 if bandwidth-bound.

THE MENTAL MODEL

Everything reduces to one idea

The GPU has one job: keep thousands of ALUs busy every clock cycle.

Everything else — memory hierarchy, warp scheduling, occupancy — exists to serve that one goal.

Coalescing	Uncoalesced access stalls the whole warp waiting for memory — ALUs idle for 600 cycles.
Bank conflicts	Serialised shared memory accesses stall the warp — ALUs wait when they could be computing.
Occupancy	Low occupancy = fewer warps to switch to when one stalls — ALUs go idle between memory ops.
Warp divergence	Serialised paths halve the active lanes — half the ALUs are masked off and wasted.
Vectorization	Fewer load instructions per byte = more instruction slots available for arithmetic ops.

FULL RECAP

The talk in one slide

01 CPU vs GPU

Latency-optimised (few powerful cores) vs throughput-optimised (thousands of simple ones). GPU hides latency through parallelism.

02 GPU Hardware

GPC → TPC → SM → Warp → Thread. The SM is where your kernel runs: CUDA cores, Tensor Cores, register file, L1/shared memory.

03 CUDA Model

Thread·Block·Grid. idx = blockIdx.x*blockDim.x + threadIdx.x. Always guard bounds. Blocks are independent — warps are scheduled.

04 Memory Hierarchy

Registers (1cy) → Shared (5cy) → L1 (28cy) → L2 (120cy) → Global/HBM (600cy). Choosing the right level IS the optimization.

05a Coalescing

Threads in a warp → consecutive addresses → 1 transaction. Stride or random → multiple transactions → bandwidth waste.

05b Bank conflicts

Shared memory: 32 banks × 4B. Same-bank access → serialised. Fix: +1 padding on power-of-2 strides.

05c Occupancy

Active warps / max warps per SM. Registers + shared memory limit it. More warps = more latency hiding = faster kernel.

05d Warp divergence

If/else inside a warp → serialised paths → throughput is reduced. Keep warps uniform.

05e Vectorization

float4 loads 16 bytes in one instruction. 4× fewer load ops. Requires 16-byte alignment. Biggest win on bandwidth-bound kernels.

WHERE TO GO NEXT

3 resources, no fluff

01

CUDA C++ Programming Guide

The authoritative reference

docs.nvidia.com/cuda/cuda-c-programming-guide

Everything in this talk — and much more — documented by NVIDIA. Read chapters on memory model and execution configuration first.

02

GPU Mode

The most active GPU programming community

discord.gg/gpumode · youtube.com/@gpumode

Weekly paper readings, CUDA lecture series, hands-on kernel optimization sessions. Discord + YouTube. Free, excellent, practical.

03

Tensara.org

Practice by writing and benchmarking real kernels

tensara.org

Competitive GPU programming problems with automatic benchmarking. The fastest way to build intuition — you see your kernel's actual GB/s and TFLOPS.

Google Colab has free GPUs — open a notebook and profile your first kernel today.

The GPU has one job.

Keep the SMs busy.

Now you know how.

Alessandro Bartolocci · Sapienza University of Rome

CUDA: Architecture, Memory & Kernel Optimization